C++结构体基础用法详解

我用**最通俗的语言+极简语法+完整示例**，给你讲解C++**结构体（struct）** 的定义和基础用法，衔接你之前学的函数、数组、`vector`，新手一看就会！

一、结构体是什么？

结构体（`struct`）是C++的**自定义数据类型**，作用：

把**多个不同类型的变量**（int/string/double等）**打包成一个整体**，用来描述一个事物的所有属性（比如：学生=姓名+年龄+成绩；书籍=书名+价格+作者）。

二、结构体的标准定义语法（核心必背）

Code block

```
// 1. 定义结构体类型（自定义数据类型）
struct 结构体名称 {
    // 成员变量（可以是任意类型：int/string/char/数组/vector等）
    数据类型 成员1;
    数据类型 成员2;
    ...
};  // ⚠️ 末尾必须加分号！新手最容易忘！
```

经典示例：定义「学生结构体」

我们把学生的**姓名(string)、年龄(int)、分数(double)** 打包：

Code block

```
#include <iostream>
#include <string>
using namespace std;

// 定义学生结构体
struct Student {
    // 成员变量
    string name;  // 姓名
    int age;      // 年龄
    double score; // 分数
```

```
};  // 必须加分号！
```

三、定义结构体变量（2种常用方式）

结构体是**类型**（和int、string一样），定义后才能用：

方式1：先定义结构体，再创建变量（最常用）

Code block

```
// 结构体类型 + 变量名
Student stu1;
```

方式2：定义结构体时，直接创建变量

Code block

```
struct Student {
    string name;
    int age;
    double score;
} stu2, stu3; // 直接定义变量stu2、stu3
```

四、访问结构体成员（核心运算符：`.`）

使用 `变量名.成员名` 访问/赋值结构体里的属性：

Code block

```
int main() {
    // 1. 创建结构体变量
    Student stu;

    // 2. 给成员赋值
    stu.name = "张三";
    stu.age = 18;
    stu.score = 95.5;

    // 3. 访问成员并打印
```

```
    cout << "姓名: " << stu.name << endl;
    cout << "年龄: " << stu.age << endl;
    cout << "分数: " << stu.score << endl;

    return 0;
}
```

五、进阶用法（结合你学过的知识）

1. 结构体数组（存储多个学生）

和普通数组用法完全一致：

Code block

```
// 定义存储3个学生的结构体数组
Student stuArr[3] = {
    {"张三", 18, 95.5},
    {"李四", 19, 88.0},
    {"王五", 20, 90.0}
};

// 遍历数组
for(int i=0; i<3; i++){
    cout << stuArr[i].name << " " << stuArr[i].score << endl;
}
```

2. 结构体 + vector（动态数组，最推荐）

结合你刚学的 `vector` ，存储任意数量的结构体：

Code block

```
#include <vector>

// 定义vector，存储Student类型
vector<Student> stuVec;

// 添加学生
Student s1 = {"小明", 18, 99};
stuVec.push_back(s1);

// 遍历vector
```

```
for(auto s : stuVec){
    cout << s.name << endl;
}
```

3. 结构体作为函数参数（传引用！）

和 `vector` 一样，**传引用&避免拷贝**，效率更高：

Code block

```
// 打印学生信息（只读，const引用）
void printStu(const Student& stu) {
    cout << stu.name << " " << stu.age << endl;
}

// 修改学生信息（读写，普通引用）
void modifyStu(Student& stu) {
    stu.score = 100;
}
```

六、完整可运行代码（整合所有用法）

Code block

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// 1. 定义结构体
struct Student {
    string name;
    int age;
    double score;
};

// 函数：打印结构体（const引用）
void printStudent(const Student& stu) {
    cout << "姓名：" << stu.name
         << " 年龄：" << stu.age
         << " 分数：" << stu.score << endl;
}

```

```
int main() {
    // 2. 单个结构体变量
    Student stu1 = {"张三", 18, 95.5};
    printStudent(stu1);

    // 3. 结构体vector
    vector<Student> stuVec;
    stuVec.push_back({"李四", 19, 88.0});
    stuVec.push_back({"王五", 20, 90.0});

    // 遍历vector
    cout << "\nvector遍历: " << endl;
    for (auto s : stuVec) {
        printStudent(s);
    }

    return 0;
}
```

运行结果

Code block

```
姓名: 张三 年龄: 18 分数: 95.5

vector遍历:
姓名: 李四 年龄: 19 分数: 88
姓名: 王五 年龄: 20 分数: 90
```

七、新手必记3个易错点

1. **结构体定义末尾必须加** `;` （语法错误重灾区）
2. 访问成员用 `.` **运算符**（变量名.成员名）
3. 函数传参用 `const 结构体&` （安全、高效）

总结

1. **定义**: `struct 名称{ 成员列表; };`
2. **变量**: 和int一样使用 `Student stu;`

3. **访问**：`stu.成员名`
4. **批量存储**：优先用 `vector<结构体>`
5. **函数传参**：用引用 `&`

（注：文档部分内容可能由 AI 生成）